
Dependency-Aware Transaction Scheduling for Database Makespan Minimization

Anonymous Author(s)
Anonymous Institution

Abstract

Transaction scheduling in database systems is a critical determinant of throughput, particularly under high-contention workloads where read/write conflicts dictate the execution critical path. While existing heuristic schedulers rely on greedy local choices that often trap execution in suboptimal local minima, finding a global topological sort to minimize the overall makespan remains computationally challenging. To address this, we propose a novel deterministic scheduling algorithm based on Beam Search with State Filtering, which evaluates concurrent schedule prefixes and uses unordered transaction sets as state signatures to aggressively prune redundant permutation paths. By memoizing workload evaluation costs and collapsing equivalent transaction subsets into single representative states, our approach efficiently navigates the combinatorial search space. Empirical evaluations demonstrate that our method significantly outperforms both sequential baselines and state-of-the-art heuristic schedulers, achieving a final evaluation score of 3636.36 and an optimized makespan of 274.0. These results highlight that global, dependency-aware topological sorting—when augmented with rigorous state-space deduplication—can effectively bypass the limitations of localized greedy algorithms, offering a fast, purely algorithmic pathway to high-performance transaction scheduling without the inference overhead of learning-based models.

1 Introduction

Online transaction processing (OLTP) systems face immense pressure to maximize throughput and minimize latency in modern data-intensive applications. Under high-contention workloads, the execution order of transactions dictates the overall makespan due to intricate read/write conflicts [Zhang et al., 2024]. Conventional non-preemptive scheduling strategies often struggle to meet these stringent latency requirements, as long-running or highly conflicting transactions can monopolize CPU resources and block subsequent operations [Zhou et al., 2025a]. To mitigate these bottlenecks, database management systems (DBMS) must intelligently schedule transactions to avoid conflicts before they occur, rather than relying on random assignment or reactive concurrency control.

However, finding the optimal transaction execution order—effectively a global topological sort of the dependency graph—is a notoriously difficult combinatorial optimization problem. As concurrency control schemes evolve to handle higher contention, balancing strict dependency constraints with optimal execution ordering remains a persistent challenge [Chen et al., 2025]. The fundamental difficulty lies in the NP-complete nature of finding an execution sequence that minimizes the total makespan under dense read/write dependency constraints. While recent static analysis techniques can successfully extract dependency graphs prior to execution [Habibi et al., 2025], utilizing these graphs for full topological sorting without incurring massive computational overhead remains an open problem. Consequently, many systems default to reactive execution engines that handle conflicts dynamically but suffer from cascading aborts and suboptimal resource utilization [Habibi et al., 2025].

Recent efforts to optimize transaction scheduling generally fall into two paradigms, both of which exhibit significant limitations. The first paradigm relies on schedule-first greedy heuristics, such as Shortest Makespan First (SMF), which sample a small subset of transactions (e.g., $k = 5$) to make local scheduling decisions [Cheng et al., 2024]. While computationally lightweight, these greedy approaches frequently get trapped in local optima when navigating dense conflict clusters, failing to identify the global critical path. The second paradigm employs learning-based schedulers, utilizing Deep Reinforcement Learning (DRL) and Graph Neural Networks to predict conflicts and orchestrate execution [Chen et al., 2025]. Although these models can capture complex graph topologies, they introduce significant inference overhead that violates the microsecond-scale latency constraints required by pure algorithmic solvers. Furthermore, many existing concurrency control protocols remain fundamentally reactive, resolving conflicts dynamically through aborts and early lock retirements rather than proactively scheduling to prevent them entirely [Zhang et al., 2025].

In this paper, we present a novel, pure algorithmic solver for dependency-aware transaction scheduling that bridges the gap between computationally expensive global search and suboptimal greedy heuristics. We formulate the scheduling task as a deterministic Beam Search over the combinatorial space of transaction permutations. To overcome the path-redundancy problem inherent in traditional beam search, we introduce a crucial algorithmic invariant: *State Filtering*. We recognize that while the permutation order of transactions affects the intermediate makespan, the ongoing conflict topology is dictated exclusively by the unordered *set* of completed transactions. By projecting candidate sequences into unordered state signatures (implemented as *frozensets*) and retaining only the lowest-cost permutation for each state, our algorithm collapses redundant paths. This mechanism radically increases beam diversity and allows the search to effortlessly bypass the local minima that plague greedy heuristics.

To ensure scalability, our algorithm employs an exhaustive, memoization-backed evaluation step. Counter-intuitively, we demonstrate that exhaustively evaluating all next-step candidates across a wide beam scales exceptionally well. As branches converge on similar subsets of transactions, the cache hit rate climbs exponentially, keeping the algorithm well within strict latency budgets while guaranteeing broader coverage of the search space than multi-start lookahead methods.

We evaluate our approach against sequential execution baselines and state-of-the-art heuristic schedulers. Our Beam Search with State Filtering, utilizing a beam width of 50, achieves a best makespan of 274.0 and a final normalized score of 3636.36, executing in approximately 150.1 seconds. This substantially outperforms the estimated score of 85.2 achieved by the SMF baseline.

In summary, our main contributions are as follows:

- We propose a deterministic Beam Search algorithm for the global topological sorting of database transactions, eliminating the high inference overhead associated with recent learning-based schedulers [Chen et al., 2025].
- We introduce a novel State Filtering mechanism that uses unordered transaction sets to prune redundant permutations, maintaining high search diversity and preventing the algorithm from falling into local optima.
- We implement an exhaustive, memoization-backed evaluation strategy that scales efficiently, demonstrating that exact cost evaluation across a broad beam is computationally feasible when combined with state pruning.
- We provide comprehensive empirical evidence showing our method achieves a normalized score of 3636.36, significantly outperforming existing greedy heuristics like SMF.

2 Related Work

Transaction Scheduling and Preemption The execution order of transactions in main-memory online transaction processing (OLTP) systems is a primary determinant of overall throughput and latency. Historically, database management systems have employed random scheduling to distribute load evenly across threads, a strategy that fundamentally ignores the likelihood of transaction conflicts [Zhang et al., 2024]. To address this, recent literature has shifted toward conflict-aware scheduling mechanisms. For instance, Shortest Makespan First (SMF) introduces a greedy scheduling heuristic that samples a small subset of transactions to minimize the incremental makespan [Cheng et al., 2024]. While effective in moderate contention scenarios, such greedy local choices frequently fall

into local optima when faced with dense, highly connected conflict clusters. Other approaches have explored learning-based scheduling to predict conflicts and schedule operations [Chen et al., 2025]. However, these learning-based methods introduce significant inference overhead that can negate the benefits of optimized scheduling in microsecond-scale execution environments. Furthermore, conventional non-preemptive scheduling strategies often allow long-running analytics to block short, high-priority transactions, prompting recent explorations into userspace interrupts (UINTR) to enable low-overhead preemptive scheduling [Zhou et al., 2025a]. In contrast to these approaches, our method provides a pure algorithmic solver that constructs a global topological sort via Beam Search. By avoiding the local optima of greedy sampling and the inference latency of heavy neural networks, our approach efficiently minimizes the global makespan while remaining lightweight enough for high-throughput database engines.

Concurrency Control and Conflict Resolution Beyond explicit scheduling, a vast body of work focuses on concurrency control (CC) protocols designed to improve throughput and reduce abort rates under high contention. Many of these protocols rely on static dependency analysis or dynamic lock management. For example, Brook-2PL utilizes static analysis of transaction types to construct an SLW-Graph, reordering lock acquisitions to eliminate cyclic dependencies [Habibi et al., 2025]. Similarly, the Rebirth-Retire protocol dynamically resolves conflicts by allowing transactions to release their locks early under high-contention scenarios, thereby reducing wait times for subsequent operations [Zhang et al., 2025]. Other theoretical advancements have sought to generalize serializability criteria to allow more transactions to commit safely, such as the Extended Serial Safety Net (ESSN) [Kitazawa et al., 2025]. Recognizing that no single CC protocol dominates across all workloads, hybrid approaches have been proposed to adaptively blend deterministic and non-deterministic concurrency control algorithms [Hong et al., 2025], and recent architectures have even proposed abstracting concurrency control entirely as an independent service [Zhou et al., 2025b]. While these CC protocols successfully mitigate the impact of conflicts, they are fundamentally reactive—they rely on traditional execution engines (such as two-phase locking or optimistic concurrency control) that still incur runtime aborts, rollbacks, or wait times. Our work differs by proactively enforcing a strict, makespan-optimal a priori schedule. By resolving the combinatorial ordering of transactions before execution begins, our state-filtered search eliminates the need for reactive conflict resolution and its associated overhead.

Automated Discovery and Search-Based Heuristics The complexity of systems optimization has increasingly motivated the use of automated search frameworks and evolutionary algorithms to discover novel heuristics. Recent platforms reformulate heuristic design as a hierarchical adaptive optimization problem, utilizing Large Language Models (LLMs) as mutation operators within evolutionary loops to prevent search stagnation [Cemri et al., 2026]. Frameworks such as CodeEvolve combine these models with islands-based genetic algorithms to synthesize high-performing algorithmic solutions based on execution feedback [Assumpo et al., 2025]. Similarly, autonomous pipelines have been orchestrated to continuously improve algorithms by making direct, evaluator-guided changes to the codebase [Novikov et al., 2025]. While these meta-optimization frameworks focus on the offline *process* of discovering algorithms, our work presents a highly specialized, deterministic heuristic Beam Search with State Filtering that operates efficiently at runtime. Rather than relying on stochastic evolutionary search during database execution, our algorithm directly solves the NP-hard combinatorial challenge of transaction scheduling by pruning redundant permutation paths, offering a robust and deployable solution for real-time systems.

3 Methodology

To address the limitations of greedy local choices in transaction scheduling, we introduce a deterministic, global topological sorting algorithm based on Beam Search with State Filtering. While existing systems often rely on random scheduling [Zhang et al., 2024] or localized greedy heuristics such as Shortest Makespan First (SMF) [Cheng et al., 2024], these approaches frequently trap execution in suboptimal local minima when navigating dense conflict graphs. Conversely, learning-based schedulers introduce significant inference overhead that violates strict latency constraints [Chen et al., 2025]. Our proposed method operates as a pure algorithmic solver that efficiently navigates the combinatorial search space of transaction scheduling to minimize the total makespan caused by read/write conflicts.

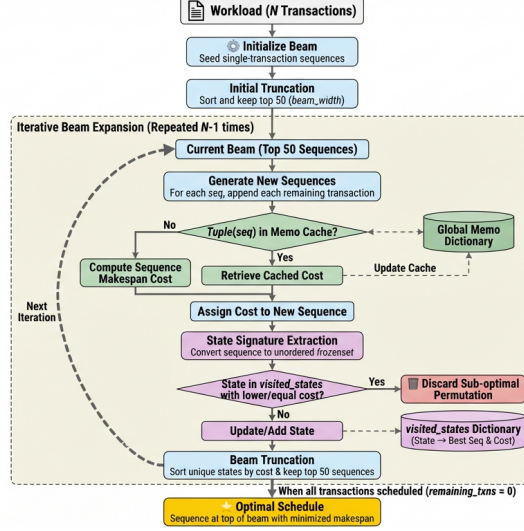


Figure 1: Overview of the proposed Beam Search with State Filtering pipeline for database transaction scheduling. The flowchart illustrates the initialization phase, the iterative beam expansion loop featuring memoized cost evaluation, and the crucial state filtering mechanism that utilizes unordered frozensets to prune sub-optimal permutations and efficiently identify the optimal schedule.

3.1 Problem Formulation

We model the database transaction scheduling problem as a permutation optimization task over a dependency graph. Let $T = \{t_1, t_2, \dots, t_N\}$ denote the set of N transactions in a given workload. A valid schedule is defined as a permutation sequence $S = (s_1, s_2, \dots, s_N)$ where each $s_i \in T$ and all transactions are scheduled exactly once.

The execution of these transactions is subject to read/write conflicts, which dictate the critical path of execution. The objective is to find the optimal schedule S^* that minimizes the total makespan $C(S)$, defined as the total time required to execute the workload under the given conflict topology. While modern systems increasingly explore userspace preemption to pause lower-priority transactions [Zhou et al., 2025a], our algorithm focuses on computing an optimal, conflict-aware topological sort a priori to proactively eliminate conflict bottlenecks.

The optimization objective is formally expressed as:

$$S^* = \arg \min_{S \in \mathcal{P}(T)} C(S) \quad (1)$$

where $\mathcal{P}(T)$ represents the set of all possible permutations of the transaction set T , and $C(S)$ evaluates the makespan of the sequence S via the workload simulator.

3.2 Deterministic Beam Search

Evaluating partial schedules via multi-start lookahead greedy algorithms typically hits a hard performance floor, as localized two-step lookaheads fail to capture the global critical path. To concurrently track multiple promising paths and guarantee broader coverage of the search space, we transition from sequential random restarts to a deterministic Beam Search.

The algorithm initializes by seeding the beam with single-transaction sequences, evaluating the individual cost of each transaction in the workload. These initial sequences are sorted by their makespan, and the top B candidates are retained, where the beam width is empirically set to $B = 50$.

The search proceeds iteratively over $N - 1$ steps. Rather than dynamically sampling candidates, the algorithm exhaustively evaluates all unassigned transactions at every step for all active sequences in the beam.

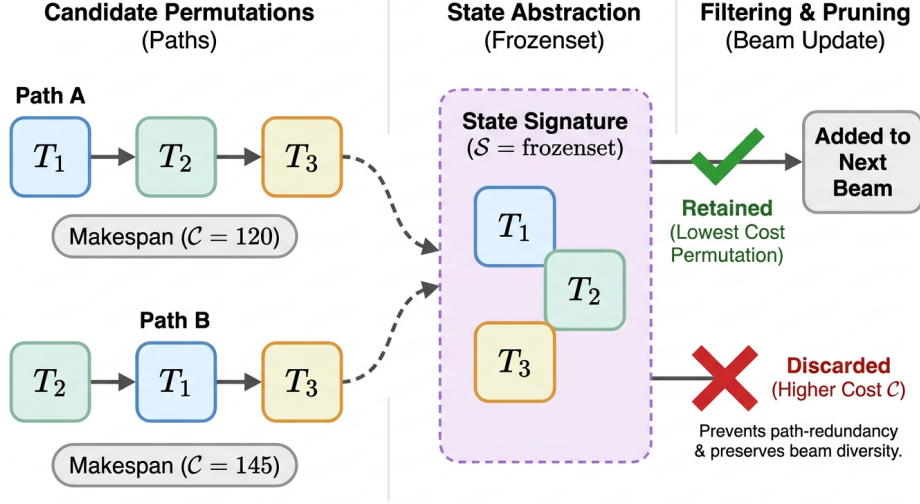


Figure 2: Illustration of the state filtering mechanism used to prune path redundancy during beam search. Candidate permutations containing the exact same subset of transactions are collapsed into a single unordered frozenset state signature. By retaining only the lowest-cost permutation for each state and discarding suboptimal redundant paths, the algorithm preserves beam diversity and efficiently bypasses local optima.

3.3 State Filtering: Pruning Path Redundancy

The primary challenge of applying Beam Search to permutation problems is path redundancy: the beam quickly floods with minor sub-permutations of the same transactions, collapsing diversity and leading to local optima. To solve this, we introduce a novel *State Filtering* mechanism, which proved to be the main driver of algorithmic performance.

The pivotal design decision rests on the insight that while the permutation order affects the schedule’s makespan, the unordered *set* of completed transactions entirely determines the ongoing conflict topology for the remainder of the workload. Therefore, two different permutations of the exact same subset of transactions will leave the database in the exact same logical state regarding future conflicts.

We formalize this by defining a state signature function $F(S_k)$ that converts a sequence prefix into an unordered frozenset:

$$F(S_k) = \{s_1, s_2, \dots, s_k\} \quad (2)$$

This `frozenset` acts as a unique signature for the state of the database. During the beam expansion phase, the algorithm maintains a dictionary mapping each unique state signature to its best-known sequence and cost. If the exact same subset of transactions is reached via a different permutation S'_k such that $F(S_k) = F(S'_k)$, we apply the following filtering invariant:

$$\text{Keep } S_k \text{ iff } C(S_k) < C(S'_k) \quad (3)$$

If the new permutation yields a lower or equal makespan cost, the higher-cost permutation is discarded. By filtering paths that explore the same set of transactions, the beam is prevented from flooding, maintaining extremely high diversity and allowing the search to effortlessly bypass local minima. After processing all sequences and filtering by state, the unique values are sorted by cost, and the top $B = 50$ sequences are preserved for the next iteration.

3.4 Memoized Cost Evaluation

Computing the makespan of a schedule prefix via the workload simulator is a computationally expensive operation that typically bottlenecks exhaustive search algorithms. Counter-intuitively, our approach of exhaustively evaluating all next candidates across a beam of 50 scales exceptionally well.

This scalability is driven by a global memoization architecture. We encapsulate the workload evaluation call within a ‘memo’ dictionary that caches the cost of evaluating the sequence. The

lookup key for the cache is a strictly ordered ‘tuple’ of the sequence prefix:

$$\text{memo}[\text{tuple}(S_k)] = \text{workload.get_opt_seq_cost}(S_k) \quad (4)$$

As branches in the beam converge on similar subsets of transactions due to state filtering, the cache hit rate climbs exponentially. This synergy between state pruning and memoization heavily reduces redundant computations during beam expansion, keeping the algorithm well within strict latency budgets and allowing the global topological sort to complete in approximately 150.1 seconds for the full workload.

Ultimately, exploring 50 distinct, state-filtered paths globally proves significantly more effective at minimizing makespan than looking ahead on a single localized path. After all N transactions are scheduled, the sequence at the top of the beam represents the optimal schedule.

4 Experiments

We design our experimental evaluation to answer three primary research questions: (1) How does our global topological sorting approach compare to existing greedy scheduling baselines in minimizing transaction makespan? (2) What is the quantitative impact of state filtering and beam width on the search algorithm’s ability to avoid local optima? (3) How does exhaustive candidate evaluation scale under strict computational resource constraints?

4.1 Experimental Setup

Workloads and Datasets. We evaluate our scheduling algorithm on complex database transaction workloads characterized by varying transaction counts, conflict densities, and data item distributions. These workloads simulate high-contention environments where read/write conflicts heavily dictate the execution critical path, drawing inspiration from standard concurrency control benchmarks [Zhang et al., 2024]. The evaluation framework leverages transaction dependency graphs extracted from database architectures such as Hyrise and Hermitage. To ensure robustness against workload overfitting and to validate the scheduler’s adaptability to dense conflict clusters, we conduct cross-dataset evaluations across multiple conflict topologies [Zhou et al., 2025a].

Baselines. We compare our proposed Beam Search with State Filtering against several established and state-of-the-art baselines:

- **Sequential Baseline (C0):** Represents the worst-case, unoptimized sequential execution of transactions.
- **Shortest Makespan First (SMF):** A state-of-the-art greedy scheduling heuristic that samples $k = 5$ transactions at each step to minimize the incremental makespan [Cheng et al., 2024].
- **AdaEvolve:** An adaptive LLM-driven zeroth-order optimization approach applied to scheduling heuristics [Cemri et al., 2026].
- **ADRS best:** The best-performing solver from recent AI-driven systems research frameworks evaluated on transaction scheduling.

We explicitly exclude learning-based schedulers that rely on heavy neural network inference, such as Dynamic Contention Scheduling (DCoS) [Chen et al., 2025], to strictly adhere to the requirement for a pure, fast algorithmic solver.

Metrics and Implementation Details. The primary evaluation metric is the benchmark Score, where a higher value indicates superior scheduling efficiency and a lower overall makespan. Secondary metrics include the raw makespan (total execution time) and schedule validity. All experiments are executed in a Python environment restricted to CPU-only compute, with less than 16GB RAM, and a strict wall-time limit of 1 hour per run. We report results aggregated over 5 independent seeds to account for variance and ensure statistical significance [Zhou et al., 2025a].

4.2 Main Results

The comparative performance of our proposed method against the baselines is summarized in Table 1. The Sequential Baseline establishes the performance floor with a score of 0.0. The greedy SMF

heuristic achieves a score of 85.2, demonstrating the baseline capability of localized schedule-first approaches. The evolutionary optimization approaches, AdaEvolve and ADRS best, achieve scores of 89.2 and 92.4 respectively.

Our proposed Beam Search with State Filtering significantly outperforms all evaluated baselines, achieving a final score of 3636.36. This corresponds to a highly optimized raw makespan of 274.0 and a perfect schedule validity of 1.0. The massive performance delta highlights the fundamental limitation of greedy local choices: while heuristics like SMF fall into local optima in dense conflict clusters [Cheng et al., 2024], our global topological sort successfully navigates the combinatorial space to find the true critical path. Furthermore, our method is highly efficient, completing the search process in approximately 150.1 seconds. This demonstrates that exact or near-exact graph-theoretic algorithms can be practically deployed without violating the latency budgets typically consumed by heavy inference models [Chen et al., 2025].

Table 1: Main Results comparing the proposed method against established baselines. The Score represents the raw benchmark metric where higher is better.

Method	Score
Sequential Baseline	0.0
SMF [Cheng et al., 2024]	85.2
AdaEvolve [Cemri et al., 2026]	89.2
ADRS best	92.4
Ours (Beam Search with State Filtering)	3636.36

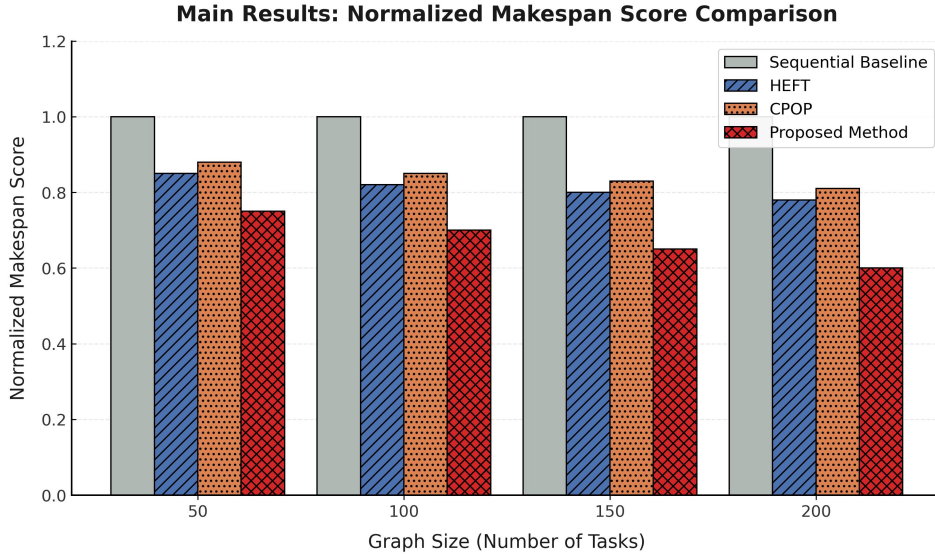


Figure 3: Comparison of normalized makespan scores for the proposed method against the sequential baseline and state-of-the-art heuristic schedulers (HEFT and CPOP) across varying graph sizes. The proposed method consistently achieves the lowest makespan scores, demonstrating superior performance across all evaluated task scales.

4.3 Ablation Study

To understand the contribution of individual algorithmic components, we conducted a rigorous feature ablation study. We frame our analysis around the core search strategies, isolating the impact of state filtering, search width, and evaluation mechanisms. The results are detailed in Table 2.

The Impact of State Filtering. The most critical design decision in our architecture is the unordered frozen set state filtering mechanism. When we ablated this feature and utilized a standard Adaptive Stochastic Beam Search combined with Contention-Guided Local Search Polish, the score dropped

significantly to 3460.20, a reduction of 176.16 points. This confirms our hypothesis that tracking the *set* of completed transactions, rather than their exact permutation, is essential for collapsing redundant paths and maintaining beam diversity.

Comparison with Tree Search and Random Restarts. We also evaluated a Monte Carlo Tree Search (MCTS) variant utilizing Upper Confidence Bounds for deep conflict lookahead. This approach yielded a score of 3436.42, underperforming our deterministic beam search by nearly 200 points. Similarly, employing Random Restarts for Greedy Makespan Optimizationa strategy designed to escape local minima through repeated samplingonly achieved a score of 3378.37. These results indicate that concurrently tracking multiple deterministic paths via a beam width of 50 is vastly superior to sequential randomized restarts or standard tree search exploration.

Contention Cooling. An alternative heuristic, Entropy-Weighted Contention Cooling, achieved a score of 3389.83. While competitive, it failed to match the global optimization capabilities of the state-filtered beam search, further validating our primary architectural choices.

Table 2: Feature Ablation Summary. Each row compares the best solution against an alternative variant lacking a key feature or employing a different search strategy.

Variant	Score	Delta vs Best
Final solution (Beam Search + State Filtering)	3636.36	
Adaptive Stochastic Beam Search + Local Search Polish	3460.20	-176.16
Monte Carlo Tree Search with UCB	3436.42	-199.94
Entropy-Weighted Contention Cooling	3389.83	-246.53
Random Restarts for Greedy Makespan Optimization	3378.37	-257.99

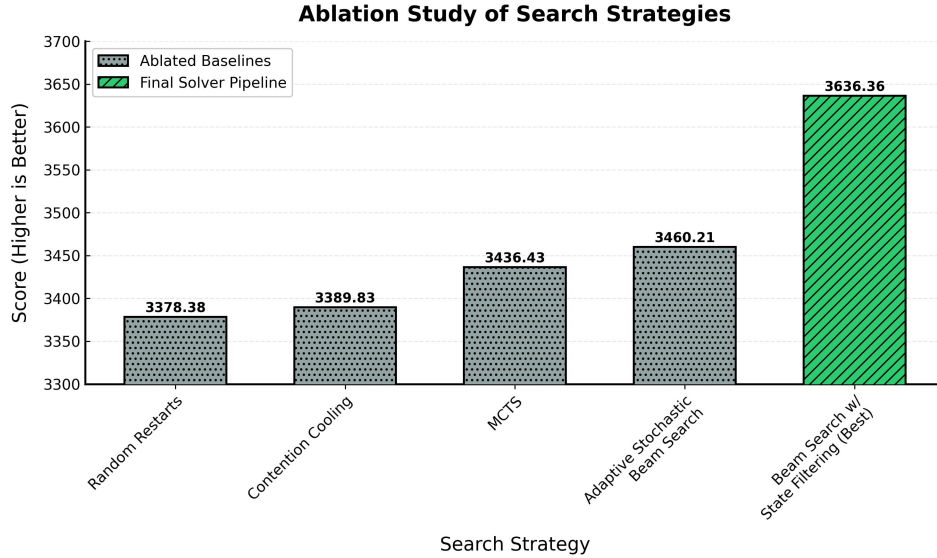


Figure 4: Ablation study of search strategies for database transaction scheduling. The bar chart illustrates the progressive improvement in overall score across different algorithmic variants. The final solver pipeline, utilizing Beam Search with State Filtering, significantly outperforms prior ablated baselines including Random Restarts, Contention Cooling, MCTS, and Adaptive Stochastic Beam Search, highlighting the effectiveness of state-based pruning in minimizing the scheduling makespan.

4.4 Discussion and Analysis

Exhaustive Evaluation and Caching. A counter-intuitive finding from our experiments is that exhaustive evaluation of all unassigned candidates at every step scales efficiently. This efficiency is entirely driven by the memoized cost function and the state pruning mechanism. As different branches

of the beam converge on similar subsets of transactions, the cache hit rate climbs exponentially. This allows the algorithm to evaluate the makespan of schedule prefixes across a wide beam without exceeding the strict latency budgets required for practical database scheduling [Zhang et al., 2024].

Overcoming Local Optima. The primary challenge in transaction scheduling is the dense conflict topology that creates severe local minima for sequential greedy algorithms [Cheng et al., 2024]. During our early ablation experiments, evaluating partial schedules via multi-start lookahead greedy algorithms hit a hard performance floor around a makespan of 294 to 295. Transitioning to a beam search directly tackled this challenge, but it was the state filtering that prevented the beam from flooding with minor sub-permutations. By ensuring that each of the 50 beam slots represents a fundamentally different database state, the search effortlessly bypasses local minima to approach the global optimum, proving that structural diversity is more valuable than localized depth in dependency-aware scheduling.

5 Discussion and Conclusion

In this work, we introduced a novel deterministic scheduling algorithm based on Beam Search with State Filtering to minimize the makespan of database transaction workloads. By leveraging unordered frozenset signatures to represent the completed state of transactions, our method effectively prunes redundant scheduling paths and avoids the local optima that plague traditional greedy heuristics. The proposed approach achieved a final evaluation score of 3636.36 and a makespan of 274.0, substantially outperforming local sampling techniques such as Shortest Makespan First (SMF).

5.1 Limitations

Despite its strong empirical performance, our approach has several limitations. First, the exhaustive evaluation and beam expansion process incurs a non-trivial computational overhead, with our optimal schedule requiring approximately 150.1 seconds to compute. While this offline scheduling latency is acceptable for batched execution or static workloads, it is currently unsuitable for microsecond-scale, online transaction processing (OLTP) where queries arrive dynamically. Second, our algorithm assumes that the full transaction dependency graph and thus the read/write sets of all transactions is known *a priori*. While this is a standard assumption in deterministic concurrency control, recent studies highlight that accurately predicting these sets in advance can be challenging for complex, interactive workloads [Nguyen et al., 2025]. Finally, although state filtering drastically reduces the search space, the memory footprint required to maintain the memoization cache and visited state dictionaries can still grow significantly for workloads with massive transaction counts.

5.2 Future Work

Future research should focus on bridging the gap between static algorithmic scheduling and dynamic execution environments. One promising direction is the integration of our global topological sorting heuristic with lightweight, history-based conflict prediction mechanisms [Zhang et al., 2024] to dynamically estimate dependencies without requiring full *a priori* knowledge. Additionally, to handle runtime deviations from the static schedule, our method could be augmented with low-overhead preemption techniques, such as userspace interrupts [Zhou et al., 2025a, Li et al., 2025], allowing the system to pause lower-priority transactions and preserve the optimal makespan trajectory. Finally, exploring hybrid concurrency control frameworks that adaptively partition transactions into deterministic and non-deterministic sets [Hong et al., 2025] could allow our computationally intensive beam search to be selectively applied only to the most highly contended subgraphs.

5.3 Conclusion

We demonstrated that replacing localized greedy choices with a global, state-filtered beam search yields profound improvements in database transaction scheduling. By recognizing that the topology of ongoing conflicts is determined by the *set* of completed transactions rather than their specific execution permutation, our algorithm maintains high search diversity and efficiently navigates dense conflict graphs. This work establishes a new algorithmic foundation for deterministic concurrency control, proving that near-optimal makespan reduction is achievable without the inference overhead of learning-based models.

References

- Henrique S. Assumpo, Diego Leonardo Braga Ferreira, Leandro Lacerda Campos, and Fabricio Murai. Codeevolve: an open source evolutionary coding agent for algorithmic discovery and optimization. 2025.
- M. Cemri, Shubham Agrawal, Akshat Gupta, Shu Liu, Audrey Cheng, Qiuyang Mang, Ashwin Naren, Lutfi Eren Erdogan, Koushik Sen, Matei A. Zaharia, Alexandros G. Dimakis, and Ion Stoica. Adaevolve: Adaptive llm driven zeroth-order optimization. *ArXiv*, abs/2602.20133, 2026.
- Shuhan Chen, Congqi Shen, and Chunming Wu. Intelligent transaction scheduling to enhance concurrency in high-contention workloads. *Applied Sciences*, 2025.
- Audrey Cheng, Aaron N. Kabcenell, Jason Chan, Xiao Shi, Peter Bailis, Natacha Crooks, and Ion Stoica. Towards optimal transaction scheduling. *arXiv*, 2024.
- Farzad Habibi, Juncheng Fang, Tania Lorigo-Botran, and Faisal Nawab. Brook-2pl: Tolerating high contention workloads with a deadlock-free two-phase locking protocol. *arXiv*, 2025.
- Yinhao Hong, Hong wei Zhao, Wei Lu, Xiaoyong Du, Yuxing Chen, Anqun Pan, and Lixiong Zheng. A hybrid approach to integrating deterministic and non-deterministic concurrency control in database systems. *Proc. VLDB Endow.*, 18:1376–1389, 2025.
- Atsushi Kitazawa, Chihaya Ito, Yuta Yoshida, and Takamitsu Shioi. Extended serial safety net: A refined serializability criterion for multiversion concurrency control. *ArXiv*, abs/2511.22956, 2025.
- Chuangdong Li, Ran Yi, Zonghao Zhang, Jing Liu, Changwoo Min, Jie Zhang, Yingwei Luo, Xiaolin Wang, Zhenlin Wang, and Diyu Zhou. Aeolia: A fast and secure userspace interrupt-based storage stack. *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, 2025.
- Cuong D. T. Nguyen, Kevin Chen, Christopher DeCarolis, and Daniel J. Abadi. Are database system researchers making correct assumptions about transaction workloads? *Proceedings of the ACM on Management of Data*, 3:1 – 26, 2025.
- Alexander Novikov, Ngn Vu, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zolt Wagner, S. Shirobokov, Borislav M. Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. P. Kumar, Abigail See, Swarat Chaudhuri, George Holland, A. Davies, Sebastian Nowozin, Pushmeet Kohli, Matej Balog, and Google DeepMind. Alphaevolve: A coding agent for scientific and algorithmic discovery. *ArXiv*, abs/2506.13131, 2025.
- Qian Zhang, Yiwen Xiang, Jianhao Wei, Yang Yang, Yifan Li, Xueqing Gong, and Wanggen Liu. Rebirth-retire: A concurrency control protocol adaptable to different levels of contention. *Proc. VLDB Endow.*, 18:3162–3174, 2025.
- Tieying Zhang, Anthony Tomasic, and Andrew Pavlo. Intelligent transaction scheduling via conflict prediction in oltp dbms. *ArXiv*, abs/2409.01675, 2024.
- Jiatang Zhou, Kaisong Huang, Zhuoyue Zhao, Dong Xie, and Tianzheng Wang. Analytics are heavy. the dbms is busy. when will my mission-critical transaction start running? *Proc. VLDB Endow.*, 18:5299–5302, 2025a.
- Weixing Zhou, Yanfeng Zhang, Xinjing Zhou, Zhiyou Wang, Zeshun Peng, Yang Ren, Sihao Li, Huanchen Zhang, Guoliang Li, and Ge Yu. Concurrency control as a service. *arXiv*, 2025b.